Sanjeev Ratnani

Computer Engineering

Company: Russell Investments India

Job Profile: Technology Intern (Application Developer)

Offer Type: Internship Only

Internship Stipend: ₹60,000 per month

CTC: Not disclosed yet

Location: Goregaon, Mumbai

Process timeline (key dates):

- **15 Feb 2025:** Application form released.
- **22 Feb 2025:** Registration deadline.
- **4–5 Mar 2025:** Shortlist calls sent out.
- **12 & 13 Mar 2025:** Interview dates offered (I attended on 12th March).
- **18 Mar 2025:** Results declared.

Resume: at the time of applying:- Russell_Resume.pdf

How applications & shortlisting worked:

- The **application and shortlisting** were entirely run by Russell Investments on **Workday**. SPIT's placement office circulated a short form so they could see who had applied, but the company did the actual screening, communication and interviews. HR communicated with students directly.
- The opening received applications from many colleges (VJTI, IIT Bombay, Thakur, Somaiya, VESIT, DJ Sanghavi, etc.) — so this was effectively a **campus-pool + off-campus hybrid**. It wasn't restricted to a single campus TPO.
- Russell had **over 9,000 applications** across approximately **5–6 internship roles**. They selected a very small number of candidates out of that pool (23 across all roles and 6 for app dev).
- Make sure to have a refined resume, a good cgpa (I'd recommend having **8+**, but try to keep **at least 7.5+** otherwise you'll miss out on a few), some achievements to get shortlisted (there were people with very good resumes but were kept on waitlisted).
- Along with a good resume also try to keep linkedin and github maintained.
  (My github & linkedin).

Interview Day schedule:

1. **Arrival & welcome (9:30 AM):** We were asked to come to their Goregaon office by 9:30 AM. The day began with HR welcoming everyone and explaining the schedule.
2. **Orientation sessions:** The HR Head gave a brief intro followed by sessions from senior leadership — the Managing Director and various Directors. Each speaker explained what Russell does, company culture, client focus, and what the tech teams build. These talks were delivered in a training room (like a PPT walkthrough) and were surprisingly detailed.
3. **Written Assessment (~11:00 AM):** After the intro sessions we took a pen-and-paper assessment (details below).
4. **Panel interviews (L1 - about 5-6 panels):** Post-assessment we moved into technical rounds — there were multiple panels running in parallel across the day. My technical panel was mid-morning/early-afternoon and lasted roughly **50-60 minutes**. Several other panels ran alongside it.
5. **Refreshments and waiting:** Between rounds we were given refreshments and asked to wait in a common area. There were several short breaks.
6. **Final rounds (L2 - 2/3 panels):** After the technical panels, a small batch of candidates (about 10 for app dev) were shortlisted for final Director-level/ hiring-manager conversations. Those rounds lasted about **25–30 minutes** each. After finishing, we were sent home around 6:00 pm and informed that the results would be shared via HR in a few days.
7. **Result:** On **18 March 2025** Russell announced the results; **six** technology interns were selected for the Application Developer role (four of us were from SPIT).

1. **Written assessment** [around 30-35 candidates on day 1 had given for app dev role]

- **Format:** Pen-and-paper, **40 minutes total**, weightage **5 marks**. Note: the test was explicitly **not eliminatory** — it contributed to evaluation but was not a strict knockout.
- **Sections & content:**
    - **10 aptitude questions** — pattern recognition, sequences, next-word / next-number style problems (IndiaBix-style).
    - **10 CS fundamentals MCQs** — typical core topics (DBMS, SQL, OS, networking basics, OOP/OO design).
    - **3 very easy coding problems — solve any 2** (but I personally solved all **three**):
        1. Valid parentheses (stack application).
        2. Missing number in an array of size n−1 containing numbers 1..n (classic sum/XOR trick).
        3. Binary-search style problem.
- **My experience:** The test was easier than I expected — I finished all sections comfortably and had time to attempt the third coding question as well. The coding questions tested basic logic, nothing too difficult.
- **Preparation tips for this round:** Regular aptitude practice (IndiaBix, common OA Questions) and brushing up on core CS topics (DBMS, OS basics, OOP) is sufficient. For the coding questions, having a command of stacks, arrays, binary search and basic bit tricks helps, you can extrapolate to covering linear structures and basic sorting algorithms.

2. **Technical interview 1** [All candidates who had given round 1]

Duration & setting: About **50-60 minutes,** weightage **10 marks,** in a room with a senior technical panel. There were **5–6 panels** running — mine was led by a very experienced interviewer (16 years of experience). The panelists were senior, technically-minded people who actually head teams.

Flow of the round (how it started):

- We began with a firm handshake and a short **personal introduction**. I handed over my resume and gave a structured 1–2 minute elevator pitch touching on my background, projects, and what I enjoy building. The interviewer asked me to go tell something that was not on my resume, followed by a deep dive into overall technical discussion.

Project deep-dive:

- I had listed a mobile app project that used **Firebase** as the backend. The panel headed into **why I chose Firebase**, how I implemented **authentication**, how I structured the database, how I handled **sync/real-time** aspects, and where I would change things for scale. They liked concrete answers (why a NoSQL approach was chosen for an app prototype, when SQL would be preferable in industry scenarios, etc.).
- Be ready to explain the **what, how, and why** for each project — not just what you implemented but design tradeoffs and alternate approaches.

System design & architecture discussion:

- They asked higher-level architecture questions: how to design an auth flow, how to approach scalability, where to put the session store, and how to handle concurrent users. They deliberately pushed to see whether I understood **tradeoffs**.
- They asked "cloud feels slow — how would you **improve performance**?" — this opened a discussion on caching, caching DB queries, denormalization vs normalization tradeoffs, and batching or asynchronous processing for heavy tasks.
- They explored my **approach to problem** statements and use cases, focusing on how I solve problems, the technical considerations I account for, and how I tackle challenges while building a project from scratch.
- My **hackathon experience**, particularly winning at SIH, impressed them due to the scale of the hackathon and demonstrated my ability to **design and plan projects** effectively, both technically and strategically.

Data structures & algorithms:

- **Two** easy to med **DSA** problems were asked (Linked Lists and Stacks were explicitly covered). For each, the panel expected: clarifying questions, a brute-force approach, and then progressive optimization to a satisfying solution with complexity analysis. I emphasise **thinking out loud** — walking through examples and edge cases, and then writing pseudo-code. They also expected short runtime/space tradeoff discussions.
- Important behavioural tip from this round: **do not pretend** to know an optimized solution even if you do — state the brute-force first, then show how you would improve it. They frequently picked words I said and asked follow-ups, sometimes on tangential points I mentioned casually.
- A brief discussion about the assessment questions was held; the questions were relatively easy, and the interviewer was very relaxed. There was a lighthearted moment when I mentioned solving all three questions, after which they asked me to **explain the approach** for just one of them.
- In some panels, if a candidate solved only two questions, the third question (from the assessment) would be asked in the interview. The assessment questions were generally easier than the interview questions, so solving and explaining one clearly in the interview left a positive impression. So if you know all 3, you could just solve 2 and take a chance that you might get asked the 3rd one in the interview.

SQL & fundamentals:

- A simple SQL query was asked (a simple JOIN query with some conditions). Beyond writing the query, they asked how to optimize it (indexes), and what a stored procedure and trigger is.

Cloud & deployment:

- We discussed **on-premise vs cloud**: I explained that cloud simplifies scaling and managed services but can have latency trade-offs; for performance-critical pieces, edge or hybrid approaches might be appropriate. They asked about packaging and deploying an app, so mention **containers, CI/CD pipelines and dockers** but I was not much aware of it so just stated my limited knowledge of those terms and didn't elaborate much.
- **! DO NOT SAY ANY THINGS THAT YOU ARE NOT AWARE OF !**
    - It is always good to say that you don't know rather than saying something wrong or the interviewer sensing that you are not much aware of the topic.

Behavioral & communication expectations in the round (key):

- Think out loud, ask for constraints, write the brute force approach and then optimize, and **write pseudo-code** once the approach is agreed upon. Test your solution with 1 example.
- If stuck, say “I need a minute to think” (they prefer structured thinking). Interviewers will often ask to justify your step; do not be defensive if they probe — it’s expected.

My personal takeaways from Round 1:

- Be crisp about your projects. When you mention an implementation detail casually, the panel will latch on to it and probe deeper — so be deliberate with your words.
- Explain tradeoffs instead of one-line answers. For example, saying “I used Firebase” should be followed by why (speed of development, managed auth, real-time DB) and when you’d not use it (complex relational joins, transaction-heavy systems).
- It was a varied experience from panel to panel, some panels had hard DSA Q solving, some evolved around resume, some went in depth with SQL, some were in detail about sorting algos, so you can’t exactly predict what can be asked but it is good to cover basics and know your resume well.

3. **Hiring Manager Round** [On day 1, about **8-10 students** had been shortlisted]

- On paper, this round was worth around **10 marks**, but the final decision largely rested with the directors and hiring managers. Although marks were assigned, I don't think the selection was purely based on scores.
- After the technical panels, Russell conducted further shortlisting: they called a small set of candidates (about **10**) for the director/hiring-manager-level conversation. Some candidates were taken aside and told they would not be moving forward. There were moments of waiting and refreshment breaks while panels and shortlists completed.
- **My final round** was with the Director at app dev (about **23 years of experience**) — this round was intentionally more conversational and lasted **around 25–30 minutes**. The Director started with a friendly tone (asked about my day, understood nerves, and deliberately relaxed the conversation). He dived on my ability to:
    - Break down **end-to-end use cases** and how I would approach them from design to deployment.
    - Explain the **SDLC** and the phases where I'd focus depending on a product's maturity.
    - Discuss **DevOps basics** — pipelines, infrastructure, monitoring, and release strategies.
    - Handle behavioral scenarios: working under deadlines, facing ambiguity, collaborating with non-tech stakeholders.
- This round evaluated **structured thinking, cultural fit, communication and ownership** more than raw coding skill.

Result & offer:

- **Results were declared on 18 March 2025.** For the Technology Intern (Application Developer) role, **6 interns** were selected; **4 of those were from SPIT**.
- The entire hiring process felt based on merit: there were students from many colleges in the applicant pool, and Russell reviewed candidates from across campuses.

My internship (brief reflection on the actual internship work):

- Now that I have completed the internship and am writing this, I can say it was **very valuable**. I worked on **two substantive problem statements** that had real ownership and impact. Weekly cross-team sessions introduced us to internal flows — not just tech teams but business and product — which helped me understand financial domain flows. Mentors were supportive and assigned meaningful responsibilities; interns even presented to Heads across offices and the MD at times. The exposure to the financial-services ecosystem, real production constraints, and presentation experience was invaluable.

What I did well (strengths that helped):

- Clear, confident **resume projects** articulated with "what/how/why" rather than surface-level bullets.
- Practiced **think-out-loud** approach during DSA — started with brute force and improved the solution.
- Paid attention to system-design tradeoffs (Firebase vs SQL, on-prem vs cloud) and explained reasoning.
- Maintained composure in the final director round (honest about nervousness, but confident on substance).
- Prepared practical examples and could tie academic knowledge to project reality.

What I could have done better (honest learnings):

- I could have prepared a more concise elevator pitch specifically tailored to the role.
- I could have rehearsed a few more behavioral stories with specific metrics (S-T-A-R with numbers).
- I could have been even more deliberate when using throwaway words (they sometimes pick up on casual mentions).
- Improve clarity on when to choose SQL over NoSQL in enterprise scenarios — have ready examples of data models to explain the choice (this is one thing I learnt really well during my internship).

What all to focus on (not just for Russell but for overall on campus tech placements):

i. Core CS fundamentals (must-do):

- Refresh **DBMS** basics: normalization, indexing, JOINs, transactions.
- Review **Operating Systems** basics.
- Networking fundamentals.
- OOPs: basic patterns and why you'd use them. (for other on campus companies, focus on doing OOPs in Java, Russell evaluated me on my overall knowledge, not a particular language)

ii. Coding & DSA (practical):

- Do arrays, linked lists, stacks/queues, trees, hashing, binary search.
- Practice medium problems on **NeetCode** or **Striver**; quality > quantity.
- When solving: ask clarifying Qs, present brute-force, optimize, write pseudo-code, test with examples, always dry run.

iii. System design / architecture:

- Practice small designs: auth flow, notifications system, basic microservice breakdown.
- Read about caching strategies (LRU caches, Redis, Hive), CDNs, load balancers, queues, and patterns for scaling. **Head First Design Patterns** helps for design patterns; for system design, learn common building blocks and tradeoffs.

iv. Cloud & DevOps basics:

- Know the basics of Cloud, how to deploy and its performance metrics.
- Be familiar with the basic terminologies of DevOps.

v. SQL practice:

- Do focused practice: **LC SQL 50** and W3Schools/TutorialsPoint for syntax and edge cases. Learn and read the basics of indexing.

vi. Aptitude & fit:

- Warm up with IndiaBix style aptitude practice; be fast and accurate for the OAs.

vii. Project preparation:

- Know your resume projects inside-out. For each project, be ready to answer: architecture diagram, deployment steps, bottlenecks encountered, alternate designs, and "if I had more time, I would…".

viii. Soft skills & behavioral:

- Prepare 6–8 STAR stories: leadership, failure & learning, conflict, impact, ownership. Use metrics where possible.
- Prepare a few smart questions tailored to the interviewer's role (not generic company questions).

ix. General longer-term preparation (CGPA, internships, hackathons):

- **CGPA:** Maintain at least 7.5+; higher CGPA helps in shortlisting. Beyond ~8 it doesn't add much (except for firms like ISS & JPMC which prefer 8.5+).
- **Internships:** Even a small/unpaid internship adds credibility — shows real-world exposure, teamwork, and practical application beyond coursework.
- **Hackathons:** Actively participate in hackathons; they build problem-solving under deadlines, give concrete projects to showcase, and often impress interviewers more than certificates.

x. Always tailor your resume according to the role and JD provided but never lie on it.

<u>Interview day strategy:</u>

Before the interview:

- Sleep well (8+ hours), eat a light breakfast, arrive **15 minutes early**.
- Review bullet points of your projects, resume, and a short list of behavioral stories.
- Carry multiple printed copies of your resume (ensure it's set to A4 size in Overleaf for proper length and accurate print layout).

During the interview:

- **Start strong** with a calm introduction — firm handshake, clear voice, concise 1–2 minute summary.
- **Think out loud.** Interviewers care about the process. Say constraints aloud; ask clarifying Qs..
- **Brute force → optimize.** If you can't find the optimal immediately, start with a simple solution and refine.
- **Write pseudo-code** and test it with sample input (edge cases included).
- **If stuck**, request a minute to organize thoughts — better than blundering through.
- Ask intelligent, role-specific questions at the end. Tailor them to the interviewer's background.

After the interview:

- Be polite and composed. Hope for the best and prepare for the worst.

Questions to ask the interviewer (tailor them):

- Have some questions prepared beforehand but there's no need to just stick to those, listen to when the interviewer introduces themselves and ask questions that would be relevant to them and their team, not just company related questions that you had looked up earlier.
- For a technical interviewer: "What are the core problems the team is solving this quarter?" / "What does success look like for an intern in just 8–12 weeks?"
- For a manager/director: "How does the team measure impact?" / "Which areas should new joiners focus on to be productive quickly?"
- Avoid generic questions like "Tell me about the company"; tie your question to the person (e.g., "You mentioned X in your talk — could you elaborate on how the team handles Y?").

Final tips & mindset:

Confidence + Humility: Be confident about what you know; be humble and honest about gaps. It's far better to say "I don't know that detail, but here's how I'd find/approach it" than to bluff.

Clarity of thought: Structured answers beat long-winded ones. Use short steps and call out assumptions.

Consistency over cramming: Daily steady practice (2–3 hours) is more effective than a single marathon.

Focus on learning concepts, reasoning, and fundamentals: Don't just chase numbers and get behind solving X number of Qs, achieving Y rank or winning Z hackathons, always keep learning as your core, pay importance to understanding from basics, success will follow.

Build projects that teach you tradeoffs: Real projects (even simple ones) let you answer "why" questions convincingly.

Health & mental prep: Sleep, minimal caffeine, and small relaxation techniques (deep breaths) before a round help dramatically.

Common pitfalls (avoid these):

- Jumping straight into code without confirming problem constraints.
- Not testing solutions with examples and edge cases.
- Overpromising or fabricating experience from your resume.
- Being too generic in behavioral answers — always add specific outcomes and numbers.
- Not asking any questions at the end or asking questions that show lack of preparation.

Useful Resources:

- **Aptitude:** IndiaBix practice problems.
- **DSA:** NeetCode / Striver.
- **SQL:** W3Schools, practice LC SQL 50 problems.
- **Design patterns & architecture:** Head First Design Patterns (intro).
- **CS Fundamentals:** Make use of GPT to generate a list of topics, and Claude to make elaborate notes, use GFG, InterviewBit for core subjects as well.
- **General interview prep:** Do refer to interview exps, look up ambition box as well.
- **Practice & exposure:** Participate in hackathons — the learning is often more valuable than winning (projects + deadlines teach you a lot).

Remember, placements can be a tough journey. It's important to realistically assess your chances, see which companies visit your college, and understand what opportunities align with your capabilities.

The main thing in on campus placements are LUCK, LUCK and LUCK!

So, if you get an opportunity for a good summer internship, it's wise to take your best shot at it and secure a strong backup. Then, you can focus on preparing for off-campus opportunities if you feel you can achieve even more. On-campus, apart from 1–2 exceptions, most companies fall within a similar range of CTC, and nothing is guaranteed. Even after solving thousands of questions, unexpected issues like technical errors can occur on d-day.

In short: aim for a good summer internship, keep a solid backup plan, and prepare for off-campus placements at a steady pace.

ALL THE BEST!

You can contact me any time for any help and guidance:

(+91 8169617362) / https://www.linkedin.com/in/sanjeev-ratnani/